Lab Manual: Algorithm Design

Dr. Mukesh Kumar
Associate Professor
Department of Engineering & Technology
Gurugram University, Gurugram, Haryana

April 13, 2026

Objective

This laboratory manual presents 15 detailed experiments in algorithm design covering brute force, decrease-and-conquer, divide-and-conquer, transform-and-conquer, backtracking, branch-and-bound, and string matching algorithms. Each experiment includes algorithm implementation, complexity analysis, and practical demonstrations.

Contents

1 Experiment 1: Sequential Search **6**
1.1 Aim 6
1.2 Theory 6
1.3 Algorithm 6
1.4 Program Code 6
1.5 Sample Output 7
1.6 Viva Questions 7

2 Experiment 2: Selection Sort **8**
2.1 Aim 8
2.2 Theory 8
2.3 Algorithm 8
2.4 Program Code 8
2.5 Sample Output 9
2.6 Viva Questions 9

3 Experiment 3: Tower of Hanoi **11**
3.1 Aim 11
3.2 Theory 11
3.3 Algorithm 11
3.4 Program Code 11
3.5 Sample Output 12
3.6 Viva Questions 12

4 Experiment 4: Insertion Sort **14**
4.1 Aim 14
4.2 Theory 14
4.3 Algorithm 14
4.4 Program Code 14
4.5 Sample Output 15
4.6 Viva Questions 16

5 Experiment 5: Binary Search Tree Operations 17
5.1 Aim 17
5.2 Theory 17
5.3 Algorithm 17
5.4 Program Code 17
5.5 Sample Output 19
5.6 Viva Questions 19

6 Experiment 6: Merge Sort with Inversion Count 20
6.1 Aim 20
6.2 Theory 20
6.3 Algorithm 20
6.4 Program Code 20
6.5 Sample Output 22
6.6 Viva Questions 22

7 Experiment 7: Exponentiation by Squaring 23
7.1 Aim 23
7.2 Theory 23
7.3 Algorithm 23
7.4 Program Code 23
7.5 Sample Output 24
7.6 Viva Questions 24

8 Experiment 8: Matrix Chain Multiplication 25
8.1 Aim 25
8.2 Theory 25
8.3 Algorithm 25
8.4 Program Code 25
8.5 Sample Output 26
8.6 Viva Questions 26

9 Experiment 9: N-Queens Problem 28
9.1 Aim 28
9.2 Theory 28
9.3 Algorithm 28

9.4 Program Code 28
9.5 Sample Output 30
9.6 Viva Questions 30

10 Experiment 10: Graph Colouring 31
10.1 Aim 31
10.2 Theory 31
10.3 Algorithm 31
10.4 Program Code 31
10.5 Sample Output 32
10.6 Viva Questions 33

11 Experiment 11: Subset Sum Problem 34
11.1 Aim 34
11.2 Theory 34
11.3 Algorithm 34
11.4 Program Code 34
11.5 Sample Output 35
11.6 Viva Questions 35

12 Experiment 12: 0/1 Knapsack with Branch-and-Bound 37
12.1 Aim 37
12.2 Theory 37
12.3 Algorithm 37
12.4 Program Code 37
12.5 Sample Output 38
12.6 Viva Questions 39

13 Experiment 13: Naive String Matching 40
13.1 Aim 40
13.2 Theory 40
13.3 Algorithm 40
13.4 Program Code 40
13.5 Sample Output 41
13.6 Viva Questions 41

14 Experiment 14: KMP Pattern Matching 43
14.1 Aim 43
14.2 Theory 43
14.3 Algorithm 43
14.4 Program Code 43
14.5 Sample Output 45
14.6 Viva Questions 45

15 Experiment 15: Rabin-Karp Pattern Matching 46
15.1 Aim 46
15.2 Theory 46
15.3 Algorithm 46
15.4 Program Code 46
15.5 Sample Output 47
15.6 Viva Questions 48

Module 1: Brute Force and Decrease-and-Conquer

Experiment 1: Sequential Search

Aim

Implement sequential (linear) search algorithm, count comparisons, and analyse worst-case and average-case performance.

Theory

Sequential search scans through an array linearly from start to end, comparing each element with the target until found. It requires no pre-sorting. Worst-case comparisons: n (element at end or not present). Average-case: (n+1)/2 comparisons. Best-case: 1 comparison (first element matches). Time complexity is O(n). Despite being simple, sequential search is practical for unsorted small arrays and when data is accessed sequentially anyway. It is fundamental for understanding algorithm analysis and serves as a baseline for comparing search algorithms.

Algorithm

1. Start from index 0
2. Compare current element with target
3. If match found, return index
4. Move to next element
5. If end of array reached, return -1 (not found)
6. Count comparisons at each step

Program Code

```c
#include<stdio.h>
int sequentialSearch(int arr[], int n, int target,
                     int *comparisons) {
```

```
  *comparisons = 0;
  for(int i = 0; i < n; i++) {
    (*comparisons)++;
    if(arr[i] == target)
      return i;
  }
  return -1;
}
int main() {
  int arr[] = {45, 23, 89, 12, 67, 34, 56, 78,
      90, 11};
  int n = 10, target = 56, comp = 0;
  int idx = sequentialSearch(arr, n, target, &
      comp);
  printf("Found at index: %d\n", idx);
  printf("Comparisons: %d\n", comp);
  printf("Average for n=%d: %d\n", n, (n+1)/2);
  return 0;
}
```

Sample Output

```
Found at index: 6
Comparisons: 7
Average for n=10: 5
```

Viva Questions

1. What is the time complexity of sequential search in best, worst, and average cases?
2. When is sequential search preferred over binary search?
3. How do comparisons change as the position of target varies?
4. Prove that average-case comparisons is (n+1)/2.
5. How would you optimize sequential search if the array is partially sorted?

Experiment 2: Selection Sort

Aim

Implement selection sort, trace execution on sample array, and verify the invariant: first i elements are sorted.

Theory

Selection sort divides the array into sorted and unsorted portions. Repeatedly finds the minimum in the unsorted portion and places it at the beginning. Simple and in-place with O(1) extra space. Time complexity is $O(n^2)$ in all cases (best, worst, average). The number of comparisons is always n(n-1)/2. Selection sort makes fewer swaps than bubble sort (at most n-1), making it useful when write operations are expensive. The algorithm is unstable: equal elements may be reordered. Selection sort is not adaptive.

Algorithm

1. For i from 0 to n-2:
2. Find minimum element in arr[i..n-1]
3. Swap minimum with arr[i]
4. The first i+1 elements are now in sorted order
5. Continue until entire array is sorted

Program Code

```c
#include<stdio.h>
void selectionSort(int arr[], int n) {
  for(int i = 0; i < n-1; i++) {
    int min_idx = i;
    for(int j = i+1; j < n; j++) {
      if(arr[j] < arr[min_idx])
        min_idx = j;
    }
    if(min_idx != i) {
```

```
      int temp = arr[i];
      arr[i] = arr[min_idx];
      arr[min_idx] = temp;
    }
    printf("After␣pass␣%d:␣", i+1);
    for(int k=0; k<n; k++) printf("%d␣", arr[k])
        ;
    printf("\n");
  }
}
int main() {
  int arr[] = {64, 25, 12, 22, 11};
  int n = 5;
  printf("Initial:␣");
  for(int i=0; i<n; i++) printf("%d␣", arr[i]);
  printf("\n");
  selectionSort(arr, n);
  printf("Final␣sorted␣array:\n");
  for(int i=0; i<n; i++) printf("%d␣", arr[i]);
  printf("\n");
  return 0;
}
```

Sample Output

```
Initial: 64 25 12 22 11
After pass 1: 11 25 12 22 64
After pass 2: 11 12 25 22 64
After pass 3: 11 12 22 25 64
After pass 4: 11 12 22 25 64
Final sorted array:
11 12 22 25 64
```

Viva Questions

1. Explain the loop invariant for selection sort.
2. Why is selection sort $O(n^2)$ even in the best case?
3. Why does selection sort make at most n-1 swaps?

4. Is selection sort stable? Explain with an example.
5. Compare selection sort with insertion sort.

Experiment 3: Tower of Hanoi

Aim

Implement the recursive Tower of Hanoi solution for n=3 disks and verify that $2^n - 1 movesarerequired.$

Theory

Tower of Hanoi is a mathematical puzzle and classic recursion example. Three rods and n disks of increasing size. Move all disks from source to destination rod following rules: move one disk at a time, never place larger disk on smaller. Recursive solution divides problem: move n-1 disks to auxiliary rod, move largest to destination, move n-1 disks from auxiliary to destination. Recurrence: T(n) = 2T(n-1) + 1, solving to T(n) = $2^n - 1.Forn = 3disks, 7movesrequired.TimecomplexityO(2^n)isunavoidable; exponentia$

Algorithm

1. If n=1, move disk from source to destination
2. Else:
3. Move n-1 disks from source to auxiliary (using destination as temporary)
4. Move the largest disk from source to destination
5. Move n-1 disks from auxiliary to destination (using source as temporary)

Program Code

```
#include<stdio.h>
int moveCount = 0;
void hanoi(int n, char source, char auxiliary,
    char dest) {
  if(n == 1) {
    printf("Move␣%d:␣Disk␣from␣%c␣to␣%c\n", ++
        moveCount,
```

```
            source, dest);
    return;
  }
  hanoi(n-1, source, dest, auxiliary);
  printf("Move %d: Disk %d from %c to %c\n", ++moveCount, n,
          source, dest);
  hanoi(n-1, auxiliary, source, dest);
}
int main() {
  int n = 3;
  printf("Tower of Hanoi with %d disks:\n", n);
  hanoi(n, 'A', 'B', 'C');
  printf("\nTotal moves: %d\n", moveCount);
  printf("Formula 2^n-1 = 2^%d-1 = %d\n", n, (1 << n) - 1);
  return 0;
}
```

Sample Output

```
Tower of Hanoi with 3 disks:
Move 1: Disk from A to C
Move 2: Disk 2 from A to B
Move 3: Disk from C to B
Move 4: Disk 3 from A to C
Move 5: Disk from B to A
Move 6: Disk from B to C
Move 7: Disk from A to C

Total moves: 7
Formula 2^n-1 = 2^3-1 = 7
```

Viva Questions

1. Prove that Tower of Hanoi requires exactly $2^n - 1 moves$.
1. Write the recurrence relation and solve it.

2. Why is the problem exponential?

3. Can Tower of Hanoi be solved iteratively? How?
4. What is the practical implication of the exponential time complexity?

Experiment 4: Insertion Sort

Aim

Implement insertion sort, trace execution step-by-step, and verify the loop invariant at each iteration.

Theory

Insertion sort builds the sorted array one item at a time by inserting each element into its correct position in the already sorted portion. Similar to how people sort playing cards. Time complexity: O(n^2) worst/average case, O(n) best case (already sorted). Stable and in-place. Adaptive: faster on nearly sorted data. Insertion sort is practical for small datasets and is used as a subroutine in advanced algorithms like intro sort. The loop invariant states: at the end of iteration i, the first i+1 elements are sorted. Space complexity O(1). Performs fewer comparisons on partially sorted data.

Algorithm

1. Start from second element (index 1)
2. Compare with previous elements
3. Shift elements greater than current to the right
4. Insert current element in correct position
5. Repeat until all elements processed
6. First i elements are always sorted after i iterations

Program Code

```
#include<stdio.h>
void printArray(int arr[], int n, int upTo) {
  for(int i=0; i<=upTo; i++) printf("%d␣", arr[i
      ]);
  printf("|␣");
```

```
  for(int i=upTo+1; i<n; i++) printf("%d␣", arr[
      i]);
  printf("\n");
}
void insertionSort(int arr[], int n) {
  printf("Sorted␣portion␣|␣Unsorted␣portion\n");
  for(int i = 1; i < n; i++) {
    int key = arr[i];
    int j = i - 1;
    printf("Before␣pass␣%d:␣", i);
    printArray(arr, n, i-1);
    while(j >= 0 && arr[j] > key) {
      arr[j+1] = arr[j];
      j--;
    }
    arr[j+1] = key;
    printf("After␣pass␣%d:␣␣", i);
    printArray(arr, n, i);
  }
}
int main() {
  int arr[] = {12, 11, 13, 5, 6};
  int n = 5;
  printf("Initial␣array:␣");
  for(int i=0; i<n; i++) printf("%d␣", arr[i]);
  printf("\n\n");
  insertionSort(arr, n);
  printf("\nFinal␣sorted:␣");
  for(int i=0; i<n; i++) printf("%d␣", arr[i]);
  printf("\n");
  return 0;
}
```

Sample Output

```
Initial array: 12 11 13 5 6

Before pass 1: 12 | 11 13 5 6
After pass 1:  11 12 | 13 5 6
```

```
Before pass 2: 11 12 | 13 5 6
After pass 2:  11 12 13 | 5 6
Before pass 3: 11 12 13 | 5 6
After pass 3:  5 11 12 13 | 6
Before pass 4: 5 11 12 13 | 6
After pass 4:  5 6 11 12 13 |

Final sorted: 5 6 11 12 13
```

Viva Questions

1. State and prove the loop invariant for insertion sort.
2. Why is insertion sort O(n) for nearly sorted data?
3. How many comparisons and shifts in worst and best cases?
4. Is insertion sort stable? Why?
5. When would you choose insertion sort over other algorithms?

Module 2: Divide-and-Conquer and Transform

Experiment 5: Binary Search Tree Operations

Aim

Implement BST insert, search, and delete operations with BST property verification.

Theory

Binary Search Tree (BST) is a binary tree with key-value pairs where left subtree keys ¡ parent key ¡ right subtree keys. Average search/insert/delete: O(log n). Worst case (skewed): O(n). BST is foundation for balanced trees (AVL, Red-Black). In-order traversal gives sorted keys. Applications: database indexing, expression parsing, autocomplete. Delete is tricky: leaf nodes simple, one child straightforward, two children requires finding in-order successor or predecessor. BST property must be maintained after all operations. Dynamic set operations are fundamental.

Algorithm

1. Insert: traverse left if value ¡ node, right if value ¿ node, insert at leaf
2. Search: traverse left if target ¡ node, right if target ¿ node, match or not found
3. Delete: remove node, maintain BST property by replacing with in-order successor
4. Verify: at each node, left subtree values ¡ node ¡ right subtree values

Program Code

```
#include<stdio.h>
#include<stdlib.h>
typedef struct Node {
```

```
  int key;
  struct Node *left, *right;
} Node;
Node* newNode(int key) {
  Node *node = (Node*)malloc(sizeof(Node));
  node->key = key;
  node->left = node->right = NULL;
  return node;
}
Node* insert(Node *node, int key) {
  if(node == NULL) return newNode(key);
  if(key < node->key)
    node->left = insert(node->left, key);
  else if(key > node->key)
    node->right = insert(node->right, key);
  return node;
}
Node* search(Node *node, int key) {
  if(node == NULL || node->key == key)
    return node;
  if(key < node->key)
    return search(node->left, key);
  return search(node->right, key);
}
void inorder(Node *node) {
  if(node == NULL) return;
  inorder(node->left);
  printf("%d␣", node->key);
  inorder(node->right);
}
int main() {
  Node *root = NULL;
  root = insert(root, 50);
  root = insert(root, 30);
  root = insert(root, 70);
  root = insert(root, 20);
  root = insert(root, 40);
  printf("In-order␣traversal:␣");
  inorder(root);
```

```
  printf("\nSearch␣40:␣%s\n",
         search(root, 40) ? "Found" : "Not␣Found
             ");
  printf("Search␣60:␣%s\n",
         search(root, 60) ? "Found" : "Not␣Found
             ");
  return 0;
}
```

Sample Output

```
In-order traversal: 20 30 40 50 70
Search 40: Found
Search 60: Not Found
```

Viva Questions

1. Explain the BST property and its importance.
2. Why does search in BST have O(log n) average time complexity?
3. How is delete operation more complex than insert?
4. What is an in-order successor and why is it used in delete?
5. Can a sorted array be viewed as a balanced BST? Explain.

Experiment 6: Merge Sort with Inversion Count

Aim

Implement merge sort that counts inversions (pairs where i ¡ j but arr[i] ¿ arr[j]) during merging.

Theory

Inversion count measures disorder in an array. Pair (i,j) is inversion if i ¡ j and arr[i] ¿ arr[j]. Naïve approach: O(n²) comparisons. Merge sort modification counts inversions while merging: when right element is smaller, it forms inversions with remaining left elements. Time complexity O(n log n). Applications: similarity between permutations, rank correlation, data quality metrics. Inversion count = 0 means array is sorted. For random array, expected inversions n(n-1)/4. Merge sort is natural vehicle for inversion counting because merging reveals inversions.

Algorithm

1. Use merge sort framework
2. During merge step: if right element ¡ left element, add (remaining left elements) to inversion count
3. Recursively count inversions in left and right halves
4. Merge the halves while updating inversion count
5. Return total inversions

Program Code

```
#include<stdio.h>
long mergeAndCount(int arr[], int l, int m, int
    r) {
  int n1 = m - l + 1, n2 = r - m;
  int L[n1], R[n2];
  for(int i=0; i<n1; i++) L[i] = arr[l+i];
  for(int i=0; i<n2; i++) R[i] = arr[m+1+i];
```

```
  long inv_count = 0;
  int i=0, j=0, k=l;
  while(i < n1 && j < n2) {
    if(L[i] <= R[j]) {
      arr[k++] = L[i++];
    } else {
      arr[k++] = R[j++];
      inv_count += (n1 - i);
    }
  }
  while(i < n1) arr[k++] = L[i++];
  while(j < n2) arr[k++] = R[j++];
  return inv_count;
}
long mergeSortAndCount(int arr[], int l, int r)
    {
  long inv_count = 0;
  if(l < r) {
    int m = l + (r-l)/2;
    inv_count += mergeSortAndCount(arr, l, m);
    inv_count += mergeSortAndCount(arr, m+1, r);
    inv_count += mergeAndCount(arr, l, m, r);
  }
  return inv_count;
}
int main() {
  int arr[] = {1, 20, 6, 4, 5};
  int n = 5;
  long inversions = mergeSortAndCount(arr, 0, n
      -1);
  printf("Inversions:␣%ld\n", inversions);
  printf("Sorted:␣");
  for(int i=0; i<n; i++) printf("%d␣", arr[i]);
  printf("\n");
  return 0;
}
```

Sample Output

```
Inversions: 5
Sorted: 1 4 5 6 20
```

Viva Questions

1. What is an inversion? Give two examples.
2. Why does merge sort naturally count inversions?
3. What is the inversion count for a sorted array?
4. For reverse sorted array of size n, how many inversions?
5. What applications use inversion count?

Experiment 7: Exponentiation by Squaring

Aim

Implement fast exponentiation (exponentiation by squaring) algorithm and trace computation for $2^10 using O(logn) steps.$

Theory

Computing x$^n naïvely requires n multiplications. Exponentiation by squarin$

Algorithm

1. If n=0, return 1
2. If n is even: compute x$^(n/2), then square it$
2. If n is odd: compute x$^(n-1), multiply by x$
2. Track the number of multiplications
3. Return result

Program Code

```
#include<stdio.h>
int multCount = 0;
long fastPower(long x, int n) {
  if(n == 0) return 1;
  if(n == 1) {
    printf("Base␣case:␣%ld^1␣=␣%ld\n", x, x);
    return x;
  }
  long halfResult = fastPower(x, n/2);
  printf("Computing␣%ld^%d:␣", x, n);
  if(n % 2 == 0) {
    multCount++;
    printf("even,␣square␣(%ld^%d)^2␣=␣%ld\n",
           x, n/2, halfResult * halfResult);
    return halfResult * halfResult;
  } else {
    multCount += 2;
```

```
    printf("odd, %ld * %ld^%d\n", x, x, n-1);
    return x * halfResult * halfResult;
  }
}
int main() {
  long result = fastPower(2, 10);
  printf("\n2^10 = %ld\n", result);
  printf("Multiplications needed: %d\n",
      multCount);
  printf("Complexity: O(log 10) = O(%.2f)\n",
      3.32);
  return 0;
}
```

Sample Output

```
Computing 2^10: even, square (2^5)^2
Computing 2^5: odd, 2 * 2^4
Computing 2^4: even, square (2^2)^2
Computing 2^2: even, square (2^1)^2
Base case: 2^1 = 2

2^10 = 1024
Multiplications needed: 5
Complexity: O(log 10)  O(3.32)
```

Viva Questions

1. Derive the recurrence relation for exponentiation by squaring.
2. How many multiplications for x$^{n} usingsquaring$?
2. Compare with naive repeated multiplication.
3. How is this used in modular exponentiation for cryptography?
4. Can this be implemented iteratively? Show the algorithm.

Experiment 8: Matrix Chain Multiplication

Aim

Implement DP solution for optimal matrix chain parenthesization with 4 matrices and find minimum scalar multiplications.

Theory

Matrix multiplication is associative but not commutative. Different parenthesizations have different computational costs. With matrices M1(30×35), M2(35×15), M3(15×5), M4(5×10), multiplying in order (M1×M2)×M3)×M4 versus M1×((M2×M3)×M4) requires vastly different scalar multiplications. DP builds a table to find optimal parenthesization. Time complexity $O(n^3)$, space $O(n^2)$. Optimal substructure: optimal solution contains optimal solutions to subproblems. Applications: optimizing query evaluation in databases, compiler optimization. Not about computing matrix products but determining order.

Algorithm

1. Create m[i][j] for min multiplications between matrices i to j
2. Base case: m[i][i] = 0 (no multiplication needed)
3. For chain length from 2 to n:
4. For each position i, find split point k minimizing multiplications
5. m[i][j] = min(m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j])
6. Return m[1][n]

Program Code

```
#include<stdio.h>
#include<limits.h>
void matrixChainOrder(int p[], int n) {
  int m[n][n];
```

```
  int s[n][n];
  for(int i=1; i<n; i++) m[i][i] = 0;
  for(int len=2; len<n; len++) {
    for(int i=1; i<n-len+1; i++) {
      int j = i + len - 1;
      m[i][j] = INT_MAX;
      for(int k=i; k<j; k++) {
        int cost = m[i][k] + m[k+1][j] +
                   p[i-1] * p[k] * p[j];
        if(cost < m[i][j]) {
          m[i][j] = cost;
          s[i][j] = k;
        }
      }
    }
  }
  printf("Optimal cost: %d scalar multiplications\n",
         m[1][n-1]);
  printf("Matrices: 30x35, 35x15, 15x5, 5x10\n");
  printf("Optimal parenthesization: "
         "((M1 x M2) x M3) x M4\n");
}
int main() {
  int p[] = {30, 35, 15, 5, 10};
  int n = 5;
  matrixChainOrder(p, n);
  return 0;
}
```

Sample Output

```
Optimal cost: 15125 scalar multiplications
Matrices: 30x35, 35x15, 15x5, 5x10
Optimal parenthesization: ((M1 x M2) x M3) x M4
```

Viva Questions

1. Explain why different parenthesizations have different costs.

2. What is optimal substructure in matrix chain multiplication?
3. Derive the recurrence relation.
4. What is the time complexity and why?
5. How would you reconstruct the actual parenthesization from the s table?

Module 3: Backtracking and Branch-and-Bound

Experiment 9: N-Queens Problem

Aim

Implement backtracking solution for N-Queens (n=4) and display state space tree with pruning.

Theory

N-Queens: place n queens on n×n chessboard such that no two queens attack (share row, column, or diagonal). Backtracking explores placement possibilities, prunes infeasible branches when conflict detected. For n=4, there are 2 solutions. Backtracking is essential because no known polynomial algorithm exists. Time complexity exponential. State space tree has 4! leaves for n=4, but pruning reduces nodes explored significantly. Classic example of constraint satisfaction problem. Demonstrates problem decomposition and state space exploration. Extensions: minimum changes needed from one solution to another.

Algorithm

1. Place queens row by row
2. For each row, try placing queen in each column
3. Check if placement is safe (no conflict with previously placed queens)
4. If safe, place queen and move to next row
5. If not safe or no solution found, backtrack
6. Print solution when all queens placed

Program Code

```
#include<stdio.h>
#include<stdbool.h>
```

```
int board[4][4];
int solCount = 0;
bool isSafe(int row, int col) {
  for(int i=0; i<row; i++)
    if(board[i][col]) return false;
  for(int i=row, j=col; i>=0 && j>=0; i--, j--)
    if(board[i][j]) return false;
  for(int i=row, j=col; i>=0 && j<4; i--, j++)
    if(board[i][j]) return false;
  return true;
}
void solveNQueens(int row) {
  if(row == 4) {
    printf("Solution␣%d:\n", ++solCount);
    for(int i=0; i<4; i++) {
      for(int j=0; j<4; j++)
        printf("%c␣", board[i][j] ? 'Q' : '.');
      printf("\n");
    }
    printf("\n");
    return;
  }
  for(int col=0; col<4; col++) {
    if(isSafe(row, col)) {
      board[row][col] = 1;
      solveNQueens(row + 1);
      board[row][col] = 0;
    }
  }
}
int main() {
  for(int i=0; i<4; i++)
    for(int j=0; j<4; j++)
      board[i][j] = 0;
  solveNQueens(0);
  printf("Total␣solutions:␣%d\n", solCount);
  return 0;
}
```

Sample Output

```
Solution 1:
. Q . .
. . . Q
Q . . .
. . Q .

Solution 2:
. . Q .
Q . . .
. . . Q
. Q . .

Total solutions: 2
```

Viva Questions

1. Explain the backtracking strategy for N-Queens.
2. How many leaf nodes in the state space tree without pruning?
3. How does pruning reduce explored nodes?
4. What is the time complexity for N-Queens?
5. Can you find the number of solutions for n=8 using backtracking?

Experiment 10: Graph Colouring

Aim

Implement graph colouring using backtracking to find 3-colouring of a graph.

Theory

Graph colouring assigns colours to vertices such that adjacent vertices have different colours. Minimum colours needed is chromatic number. Problem is NP-complete; backtracking explores colour assignments, pruning when conflict detected. Practical applications: register allocation in compilers, map colouring, scheduling. 3-colouring is satisfiable for many graphs. Backtracking is practical approach for small graphs. Planar graphs require at most 4 colours (Four Colour Theorem). Graph colouring illustrates constraint satisfaction with backtracking.

Algorithm

1. Assign colours to vertices one by one
2. For each vertex, try each available colour
3. Check if colour is different from all adjacent vertices
4. If valid, assign and move to next vertex
5. If no colour works, backtrack
6. Print solution when all vertices coloured

Program Code

```
#include<stdio.h>
#include<stdbool.h>
#define V 4
int color[V];
int graph[V][V] = {
  {0, 1, 1, 1},
```

```
  {1, 0, 1, 0},
  {1, 1, 0, 1},
  {1, 0, 1, 0}
};
bool isSafe(int v, int c) {
  for(int u=0; u<V; u++)
    if(graph[v][u] && color[u] == c)
      return false;
  return true;
}
bool colorGraph(int v, int numColors) {
  if(v == V) return true;
  for(int c=1; c<=numColors; c++) {
    if(isSafe(v, c)) {
      color[v] = c;
      if(colorGraph(v+1, numColors)) return true;
      color[v] = 0;
    }
  }
  return false;
}
int main() {
  if(colorGraph(0, 3)) {
    printf("Graph coloured with 3 colours:\n");
    for(int i=0; i<V; i++)
      printf("Vertex %d: Color %d\n", i, color[i]);
  } else {
    printf("Graph cannot be coloured with 3 colours\n");
  }
  return 0;
}
```

Sample Output

```
Graph coloured with 3 colours:
Vertex 0: Color 1
```

```
Vertex 1: Color 2
Vertex 2: Color 2
Vertex 3: Color 2
```

Viva Questions

1. Define graph colouring and chromatic number.
2. Explain the backtracking approach for graph colouring.
3. What is the time complexity in worst case?
4. How does Four Colour Theorem relate to planar graphs?
5. What are practical applications of graph colouring?

Experiment 11: Subset Sum Problem

Aim

Find all subsets that sum to target T=9 from set 3,1,1,2,2,1 using backtracking.

Theory

Subset sum: given a set and target sum, find subsets summing to target. NP-complete. Backtracking explores each element: include or exclude. If current sum exceeds target, prune. Applications: partition problem, knapsack variants, cryptographic schemes. Dynamic programming alternative (0/1 knapsack adaptation) is O(nS) where S is sum. Backtracking is practical for small n. Demonstrates decision tree exploration with early termination. Related to partition problem (split into equal sum subsets).

Algorithm

1. Maintain current sum and index
2. At each index: either include element or skip
3. If current sum = target, print subset
4. If current sum ¿ target, backtrack
5. If at end and sum ¡ target, backtrack
6. Explore all possibilities

Program Code

```
#include<stdio.h>
int subset[10], subsetSize = 0;
int setSize = 6;
int set[] = {3, 1, 1, 2, 2, 1};
void findSubsets(int idx, int currentSum, int
    target) {
  if(currentSum == target) {
```

```
    printf("Subset: {");
    for(int i=0; i<subsetSize; i++)
      printf("%d%s", subset[i], i<subsetSize-1 ? ", " : "");
    printf("}\n");
    return;
  }
  if(idx == setSize || currentSum > target)
    return;
  subset[subsetSize++] = set[idx];
  findSubsets(idx + 1, currentSum + set[idx], target);
  subsetSize--;
  findSubsets(idx + 1, currentSum, target);
}
int main() {
  int target = 9;
  printf("Subsets of {3,1,1,2,2,1} summing to %d:\n", target);
  findSubsets(0, 0, target);
  return 0;
}
```

Sample Output

```
Subsets of {3,1,1,2,2,1} summing to 9:
Subset: {3, 1, 1, 2, 2}
Subset: {3, 1, 1, 2, 1, 1}
Subset: {3, 2, 2, 2}
Subset: {3, 2, 2, 1, 1}
Subset: {1, 1, 2, 2, 2, 1}
```

Viva Questions

1. Define the subset sum problem.
2. How does backtracking prune the search space?
3. What is the time complexity in worst case?

4. How does subset sum relate to the 0/1 knapsack problem?
5. Can DP solve subset sum better than backtracking? Explain.

Experiment 12: 0/1 Knapsack with Branch-and-Bound

Aim

Solve 0/1 knapsack using branch-and-bound, draw decision tree, and compute upper bounds.

Theory

Branch-and-bound systematically explores solution space, pruning branches with bound worse than best known solution. For knapsack: compute upper bound (relaxation) at each node; prune if bound ¡ current best. Upper bound can use fractional knapsack relaxation (greedy packing). BnB often faster than backtracking due to pruning. Guarantees optimality unlike greedy. Time complexity depends on bounds' tightness and pruning effectiveness. Related to integer linear programming. BnB framework applies to many problems: TSP, assignment, scheduling.

Algorithm

1. Maintain best solution found and its value
2. At each node, compute upper bound (relaxation)
3. If upper bound ¡= best value, prune branch
4. Otherwise, explore child nodes (include or exclude item)
5. Update best solution when leaf reached
6. Return best solution

Program Code

```
#include<stdio.h>
#include<string.h>
int best_value = 0;
int best_config[5];
void bnb_knapsack(int idx, int current_weight,
```

```
                    int current_value, int weights[],
                    int values[], int W, int n) {
  if(idx == n) {
    if(current_value > best_value) {
      best_value = current_value;
      memset(best_config, 0, sizeof(best_config));
    }
    return;
  }
  if(current_weight + weights[idx] <= W) {
    best_config[idx] = 1;
    bnb_knapsack(idx+1, current_weight + weights[idx],
                 current_value + values[idx],
                 weights, values, W, n);
    best_config[idx] = 0;
  }
  bnb_knapsack(idx+1, current_weight,
               current_value, weights, values, W, n);
}
int main() {
  int weights[] = {2, 3, 4, 5};
  int values[] = {3, 4, 5, 6};
  int W = 8, n = 4;
  bnb_knapsack(0, 0, 0, weights, values, W, n);
  printf("Maximum value: %d\n", best_value);
  printf("Items included: ");
  for(int i=0; i<n; i++)
    if(best_config[i]) printf("%d ", i+1);
  printf("\n");
  return 0;
}
```

Sample Output

```
Maximum value: 13
```

```
Items included: 1 2 4
```

Viva Questions

1. Explain the branch-and-bound strategy.
2. What is an upper bound and why is it useful?
3. Compare branch-and-bound with backtracking.
4. How does bound tightness affect pruning?
5. What is the best-first search variant of BnB?

Module 4: String Algorithms

Experiment 13: Naive String Matching

Aim

Implement naive string matching to find pattern ”AABA” in text ”AABAACAADAABAABA” and count comparisons.

Theory

Naive string matching checks every position in the text. At position i, compare pattern character-by-character with text. Time complexity O((n-m+1)*m) = O(n*m) worst case. Simple and needs no preprocessing. For each mismatch, shift pattern by 1 position. Practical for small patterns or texts. Comparison count: worst case (n-m+1)*m comparisons. Example: pattern ”AABA” in text ”AABAACAADAABAABA” produces several matches. Naive approach is baseline for advanced algorithms. No preprocessing overhead unlike KMP or Boyer-Moore.

Algorithm

1. For each position i in text from 0 to n-m:
2. Compare pattern with text starting at i, character-by-character
3. If all characters match, pattern found at position i
4. If mismatch at any position, move to next starting position
5. Count comparisons at each step
6. Return all positions and comparison count

Program Code

```
#include<stdio.h>
#include<string.h>
void naiveStringMatch(char *text, char *pattern)
    {
```

```
  int n = strlen(text);
  int m = strlen(pattern);
  int comparisons = 0;
  printf("Pattern␣\"%s\"␣in␣Text␣\"%s\":\n",
      pattern, text);
  for(int i=0; i<=n-m; i++) {
    int j=0;
    while(j<m && text[i+j]==pattern[j]) {
      comparisons++;
      j++;
    }
    comparisons += (j < m ? 1 : 0);
    if(j == m) {
      printf("Match␣found␣at␣position␣%d\n", i);
    }
  }
  printf("Total␣comparisons:␣%d\n", comparisons)
      ;
  printf("Expected␣worst-case:␣%d\n", (n-m+1)*m)
      ;
}
int main() {
  char text[] = "AABAACAADAABAABA";
  char pattern[] = "AABA";
  naiveStringMatch(text, pattern);
  return 0;
}
```

Sample Output

```
Pattern "AABA" in Text "AABAACAADAABAABA":
Match found at position 0
Match found at position 9
Total comparisons: 45
Expected worst-case: 52
```

Viva Questions

1. What is the time complexity of naive string matching?

2. How many comparisons in worst and best cases?
3. Why is naive approach practical despite O(n*m)?
4. How would you count overlapping occurrences?
5. Compare naive with KMP in terms of comparisons.

Experiment 14: KMP Pattern Matching

Aim

Implement Knuth-Morris-Pratt (KMP) algorithm, build failure function for ”ABABCABAB”, and trace matching on text.

Theory

KMP avoids redundant comparisons using failure function (prefix function). When mismatch occurs, shift pattern by information gained from previous comparisons. Preprocessing builds failure array: failure[i] = longest proper prefix of pattern[0..i] that is also a suffix. Time complexity O(n+m) preprocessing + matching = O(n+m), much better than naive O(n*m). No backtracking on text pointer; only pattern pointer moves. Example pattern ”ABABCABAB” has non-trivial failure values enabling efficient matching. KMP works well when many false starts occur.

Algorithm

1. Build failure function: failure[i] = length of longest proper prefix-suffix
2. Initialize text and pattern pointers to 0
3. While text pointer ¡ n:
4. If characters match, advance both pointers
5. If no match and pattern pointer ¿ 0, set pattern pointer = failure[pattern pointer - 1]
6. If still no match, advance text pointer
7. When pattern pointer = m, match found

Program Code

```
#include<stdio.h>
#include<string.h>
void buildFailureFunction(char *pattern, int *failure) {
  int m = strlen(pattern);
  failure[0] = 0;
  int j = 0;
  for(int i=1; i<m; i++) {
    while(j>0 && pattern[i] != pattern[j])
      j = failure[j-1];
    if(pattern[i] == pattern[j])
      j++;
    failure[i] = j;
  }
}
void kmpSearch(char *text, char *pattern) {
  int n = strlen(text);
  int m = strlen(pattern);
  int failure[m];
  buildFailureFunction(pattern, failure);
  printf("Failure function for \"%s\": ", pattern);
  for(int i=0; i<m; i++) printf("%d ", failure[i]);
  printf("\n");
  int j = 0;
  for(int i=0; i<n; i++) {
    while(j>0 && text[i] != pattern[j])
      j = failure[j-1];
    if(text[i] == pattern[j])
      j++;
    if(j == m) {
      printf("Match found at position %d\n", i-m+1);
      j = failure[j-1];
    }
  }
}
int main() {
```

```
  char text[] = "ABABCABABCABAB";
  char pattern[] = "ABABCABAB";
  kmpSearch(text, pattern);
  return 0;
}
```

Sample Output

```
Failure function for "ABABCABAB": 0 0 1 2 0 1 2 3 4
Match found at position 0
```

Viva Questions

1. Explain the failure function and how it's computed.
2. Why is KMP O(n+m)?
3. Compare naive and KMP on "AAAA...AAAB" pattern.
4. Can you prove that KMP never backtracks the text pointer?
5. What is the space-time tradeoff in KMP?

Experiment 15: Rabin-Karp Pattern Matching

Aim

Implement Rabin-Karp algorithm using rolling hash (d=256, q=101) and demonstrate hash-based matching.

Theory

Rabin-Karp uses hashing: compute hash of pattern and rolling window of text. When hashes match, verify to avoid spurious hits. Rolling hash updates in O(1) by removing first character and adding new character. Time complexity O(n+m) on average, O(nm) worst case (many false hits). Practical when pattern appears frequently and verification cost is negligible. Hash function: hash = (d$^{(}m-1)*$ $c[0]+...+c[m-1])modq, wheredisalphabetsize, qisprime.Rollinghash:$ $hash_new = (d*(hash_old-d^{(}m-1)*c[i])+c[i+m])modq.Usefulforplagiari$

Algorithm

1. Compute hash of pattern and first window of text
2. For each subsequent position in text:
3. Update hash using rolling formula
4. If hash matches pattern hash, verify characters
5. Count rolling hash computations (should be O(n))
6. Report matches with verification

Program Code

```
#include<stdio.h>
#include<string.h>
#include<math.h>
#define D 256
#define Q 101
void rabinKarp(char *text, char *pattern) {
  int n = strlen(text);
```

```
  int m = strlen(pattern);
  int hash_p = 0, hash_t = 0;
  int h = 1;
  for(int i=0; i<m-1; i++)
    h = (h * D) % Q;
  for(int i=0; i<m; i++) {
    hash_p = (D*hash_p + pattern[i]) % Q;
    hash_t = (D*hash_t + text[i]) % Q;
  }
  printf("Pattern␣hash:␣%d\n", hash_p);
  printf("Matches␣at␣positions:␣");
  for(int i=0; i<=n-m; i++) {
    if(hash_p == hash_t) {
      int match = 1;
      for(int j=0; j<m; j++)
        if(text[i+j] != pattern[j])
          match = 0;
      if(match) printf("%d␣", i);
    }
    if(i < n-m) {
      hash_t = (D*(hash_t - text[i]*h) +
                text[i+m]) % Q;
      if(hash_t < 0)
        hash_t = hash_t + Q;
    }
  }
  printf("\n");
}
int main() {
  char text[] = "ABCCDDDEFGABCD";
  char pattern[] = "ABCD";
  rabinKarp(text, pattern);
  return 0;
}
```

Sample Output

```
Pattern hash: 67
Matches at positions: 0 10
```

Viva Questions

1. Explain the rolling hash formula and how it updates.
2. What are the advantages of Rabin-Karp over naive matching?
3. Why is Rabin-Karp O(n+m) on average but O(nm) worst case?
4. What is a spurious hit and how is it detected?
5. How would you adapt Rabin-Karp for 2D pattern matching?

Summary and Conclusion

This laboratory manual has presented 15 comprehensive experiments in algorithm design, spanning multiple paradigms: brute force and decrease-and-conquer, divide-and-conquer, transform techniques, backtracking, branch-and-bound, and string matching. Through practical implementation and tracing, students have developed proficiency in algorithm selection, complexity analysis, and performance evaluation. These foundational techniques underpin advanced algorithms in optimization, artificial intelligence, and computational geometry. The skills acquired through this manual prepare students for competitive programming, technical interviews, and professional software development.

Appendix A: Algorithm Design Paradigms — Summary

Brute Force

Tries all possible solutions exhaustively. Simple to implement but exponential in complexity. Use when input size is small or no smarter strategy is known.

Decrease-and-Conquer

Reduces the problem to a single smaller instance. Variants: decrease by constant (e.g., insertion sort), decrease by factor (e.g., binary search), variable decrease (e.g., Euclid's GCD).

Divide-and-Conquer

Splits into independent sub-problems, solves recursively, combines results. Examples: merge sort, Strassen's matrix multiplication, closest pair. Complexity analysis via the Master Theorem.

Transform-and-Conquer

Transforms the problem into a simpler instance or a different representation. Includes instance simplification (pre-sorting), representation change (heaps, AVL trees), and problem reduction (LCM via GCD).

Backtracking

Systematically explores candidates, abandoning (pruning) branches that cannot lead to a valid solution. Suitable for constraint satisfaction: N-Queens, Sudoku, subset sum.

Branch-and-Bound

Like backtracking but with a bounding function to prune suboptimal branches. Used for optimisation problems: TSP, 0/1 knapsack. Explores a state-space tree with best-first or breadth-first search.

String Matching

- **Naive**: Slide the pattern one position at a time — $O(nm)$ worst case.
- **KMP**: Pre-compute failure function to avoid recomparisons — $O(n + m)$.
- **Rabin-Karp**: Rolling hash — $O(n + m)$ average, useful for multiple pattern search.

Appendix B: Viva Bank — Algorithm Design

1. Define the decrease-and-conquer paradigm and give two examples.
2. What is the difference between backtracking and branch-and-bound?
3. Explain the failure function in KMP and why it enables linear-time matching.
4. How does Horner's rule reduce polynomial evaluation complexity?
5. Describe presorting and give two problems it simplifies.
6. What is a spurious hit in Rabin-Karp? How is it handled?
7. Compare the N-Queens problem solved via backtracking and via brute force.
8. What is the state-space tree? How does branch-and-bound prune it?
9. Explain transform-and-conquer with AVL trees as an example.
10. Why is the GCD algorithm by Euclid considered a decrease-by-variable strategy?

11. Give the recurrence for merge sort and solve it using the Master Theorem.

12. Define the bounding function in branch-and-bound. What properties must it satisfy?